Integrating Quran Font Rendering

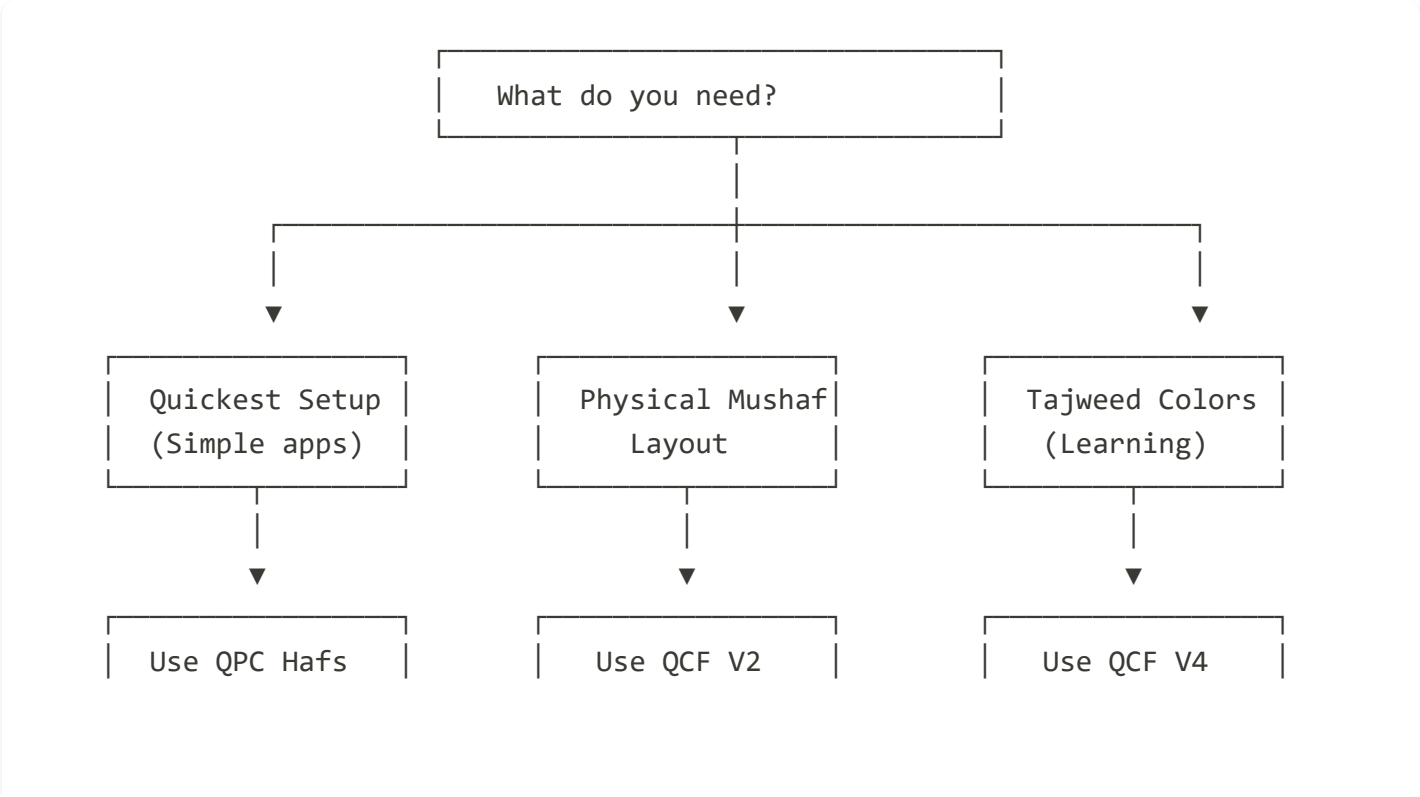
Overview

This guide explains how to integrate Quran text rendering in your application using the same fonts and techniques used on Quran.com. You'll learn how to:

- Fetch verse data with the correct API parameters for font rendering
- Load and apply Quran fonts
- Render Arabic text with proper styling
- Handle different script types (Madani, IndoPak, Uthmani, Tajweed)

TL;DR: Quick Decision Guide

Choose your path based on your needs:



```
│   (Unicode)      │        │  (Glyph-based)  │        │   (Tajweed)     │
└──────────────────┘        └─────────────────┘        └─────────────────┘
```

If you want...	Use this font	API field
Fastest implementation	QPC Hafs	`text_qpc_hafs`
South Asian script	IndoPak	`text_indopak`
Pixel-perfect Mushaf	QCF V2	`code_v2` + `page_number`
Tajweed color rules	QCF V4	`code_v2` + `page_number`

Quick Start

Simplest Implementation (Unicode Font)

If you want the quickest setup, use Unicode fonts (QPC Hafs). No page-based font loading required:

```
<!DOCTYPE html>
<html lang="ar" dir="rtl">
<head>
  <style>
    @font-face {
      font-family: 'UthmanicHafs';
      src:
url('https://verses.quran.foundation/fonts/quran/hafs/uthmanic_hafs/UthmanicHafs1Ver
format('woff2'),

url('https://verses.quran.foundation/fonts/quran/hafs/uthmanic_hafs/UthmanicHafs1Ver
format('truetype');
      font-display: swap;
    }

    .quran-text {
      font-family: 'UthmanicHafs', 'Traditional Arabic', serif;
      font-size: 28px;
      line-height: 2;
      direction: rtl;
```

```
      text-align: right;
    }
  </style>
</head>
<body>
  <div class="quran-text" id="verse"></div>

  <script>
    // NOTE: In production, fetch token server-side and pass to client
    // Never expose client_secret in browser code!
    const API_BASE = 'https://apis.quran.foundation/content/api/v4';
    const accessToken = 'YOUR_ACCESS_TOKEN'; // Get from your server
    const clientId = 'YOUR_CLIENT_ID';

    fetch(`${API_BASE}/verses/by_key/1:1?words=true&word_fields=text_uthmani,text_qp
{
      headers: {
        'x-auth-token': accessToken,
        'x-client-id': clientId
      }
    })
      .then(res => res.json())
      .then(data => {
        const text = data.verse.words.map(w => w.text_qpc_hafs).join(' ');
        document.getElementById('verse').textContent = text;
      });
  </script>
</body>
</html>
```

> ⚠️ **Security Note:** The example above shows client-side code for simplicity. In production, always obtain access tokens on your server and proxy API requests to avoid exposing credentials.

Understanding Font Types

Font Categories

Quran.com supports two categories of fonts:

Category	Description	Fonts	Pros	Cons
QCF (Glyph-Based)	Special glyph codes mapped to custom fonts	V1, V2, V4 (Tajweed)	Pixel-perfect Mushaf rendering	Requires per-page font loading
Unicode	Standard Arabic Unicode text	QPC Hafs, Uthmani, IndoPak	Simple to implement	Standard text rendering

Available Fonts

Font Name	API Field	Mushaf ID	Type	Best For
QCF V1	`code_v1`	2	Glyph-based	Traditional Madani Mushaf look
QCF V2	`code_v2`	1	Glyph-based	Modern Madani Mushaf (recommended)
QCF V4 Tajweed	`code_v2`	19	Glyph-based	Colored Tajweed rules
QPC Hafs	`text_qpc_hafs`	5	Unicode	Use with the `UthmanicHafs` font family (`UthmanicHafs1Ver18` files)
Uthmani	`text_uthmani`	4	Unicode	Standard Uthmani text
IndoPak	`text_indopak`	3, 6, 7	Unicode	South Asian users

Choosing a Font

Choose QCF V2 if:

- You want pixel-perfect Mushaf rendering

- You're building a dedicated Quran app
- You can handle dynamic font loading

Choose QPC Hafs if:

- You want simple implementation
- You're embedding Quran verses in another app
- You need quick setup

Choose Tajweed V4 if:

- You want to display Tajweed color rules
- You can handle additional theme complexity

API Parameters for Font Rendering

This section covers the specific API parameters needed for font rendering. For complete API documentation including authentication, endpoints, and general usage, see the Content APIs Quickstart Guide.

Essential Parameters

Parameter	Description	Example
`words`	Include word-level data	`words=true`
`word_fields`	Which text fields to return	`word_fields=code_v2,text_qpc_hafs,text_uthmani_simple`
`mushaf`	Mushaf layout/format ID	`mushaf=1` (QCF V2)

Parameter	Description	Example
`translations`	Translation resource IDs to include	`translations=131,95` (comma-separated)

Word Fields for Font Rendering

The key word fields needed for font rendering:

Field	Description	Required For
`code_v1`	QCF V1 glyph codes	V1 font rendering
`code_v2`	QCF V2 glyph codes	V2 or V4 font rendering
`text_qpc_hafs`	QPC Hafs Unicode text	Unicode font rendering
`text_indopak`	IndoPak script text	IndoPak font rendering

📚 **Full field reference:** Word-level Fields Documentation

Understanding Character Types

Each word in the API response has a `char_type_name` field indicating its type:

Type	Description	Rendering Notes
`word`	Regular Quranic word	Standard rendering
`end`	Verse end marker (۝)	Always use `UthmanicHafs` font
`pause`	Pause/stop mark	May skip rendering in some views
`sajdah`	Prostration marker	Special styling may apply

Type	Description	Rendering Notes
`rub-el-hizb`	Quarter Hizb marker	Special styling may apply

> ⚠️ **Important:** For `end` markers, always use the Unicode font (`UthmanicHafs`), not QCF fonts, as the verse number glyphs render better with the Unicode font.

API Request Examples

For QCF V2 (Glyph-Based)

```
curl -X GET "https://apis.quran.foundation/content/api/v4/verses/by_chapter/1?
words=true&word_fields=code_v2,text_qpc_hafs&mushaf=1" \
  -H "x-auth-token: YOUR_ACCESS_TOKEN" \
  -H "x-client-id: YOUR_CLIENT_ID"
```

For QPC Hafs (Unicode)

```
curl -X GET "https://apis.quran.foundation/content/api/v4/verses/by_chapter/1?
words=true&word_fields=text_qpc_hafs" \
  -H "x-auth-token: YOUR_ACCESS_TOKEN" \
  -H "x-client-id: YOUR_CLIENT_ID"
```

For Tajweed V4

```
curl -X GET "https://apis.quran.foundation/content/api/v4/verses/by_chapter/1?
words=true&word_fields=code_v2,text_qpc_hafs&mushaf=19" \
  -H "x-auth-token: YOUR_ACCESS_TOKEN" \
  -H "x-client-id: YOUR_CLIENT_ID"
```

API Response Structure

```
{
    "verses": [
        {
            "id": 1,
            "verse_number": 1,
            "verse_key": "1:1",
            "hizb_number": 1,
```

```
"rub_el_hizb_number": 1,
"ruku_number": 1,
"manzil_number": 1,
"sajdah_number": null,
"page_number": 1,
"juz_number": 1,
"words": [
    {
        "id": 1,
        "position": 1,
        "audio_url": "wbw/001_001_001.mp3",
        "char_type_name": "word",
        "code_v2": "ﭣ",
        "text_qpc_hafs": "بِسۡمِ",
        "page_number": 1,
        "line_number": 2,
        "text": "ﭣ",
        "translation": {
            "text": "In (the) name",
            "language_name": "english"
        },
        "transliteration": {
            "text": "bis'mi",
            "language_name": "english"
        }
    },
    {
        "id": 2,
        "position": 2,
        "audio_url": "wbw/001_001_002.mp3",
        "char_type_name": "word",
        "code_v2": "ﱃ",
        "text_qpc_hafs": "ٱللَّهِ",
        "page_number": 1,
        "line_number": 2,
        "text": "ﱃ",
        "translation": {
            "text": "(of) Allah",
            "language_name": "english"
        },
        "transliteration": {
            "text": "l-lahi",
            "language_name": "english"
        }
```

```
                },
                ....
            ]
        },
        ...
    ],
    "pagination": {
        "per_page": 10,
        "current_page": 1,
        "next_page": null,
        "total_pages": 1,
        "total_records": 7
    }
}
```

View Modes: Reading View vs Translation View

When building a Quran application, you'll typically implement one of two view modes:

Aspect	Reading View	Translation View
Layout Unit	Mushaf page (lines)	Single verse
Data Grouping	Words → Lines → Pages	Verses (no grouping)
Line Construction	Required	Not applicable
Use Case	Physical Mushaf layout	Study with translations

Reading View: From API Response to Rendered Page

The API returns **verses**, but Reading View needs **lines** to match the physical Mushaf layout.

Why? A single Mushaf line often contains words from multiple verses. You must group by line, not by verse.

Word Layout Properties:

Each word includes:

- `page_number`: Which Mushaf page (1-604)
- `line_number`: Which line on that page (1-15 typically)
- `position`: Word order within the verse

> **Note:** The API returns words in correct Mushaf order, so no sorting is needed—just group by line.

Data Transformation (Pseudo-code):

```
// Step 1: Extract all words from all verses (already in correct order)
words = verses.flatMap(verse => verse.words)

// Step 2: Group words by page and line (order is preserved)
lines = groupBy(words, word => `page-${page_number}-line-${line_number}`)
// Result: { "page-1-line-1": [words], "page-1-line-2": [words], ... }

// Step 3: Render each line, then render words within each line
for each line in lines:
    for each word in line:
        render word with font based on page_number
```

Key Points:

1. **Cross-Verse Lines:** One line may contain words from multiple verses
2. **Font Loading:** For QCF fonts, `page_number` determines which font file to load

Complete Reading View Example (QCF V2)

```
<!DOCTYPE html>
<html lang="ar" dir="rtl">
<head>
  <meta charset="UTF-8">
  <title>Quran Reading View - Glyph-Based Rendering</title>
  <style>
    @font-face {
      font-family: 'UthmanicHafs';
      src:
url('https://verses.quran.foundation/fonts/quran/hafs/uthmanic_hafs/UthmanicHafs1Ver
format('woff2');
      font-display: swap;
    }
```

```css
.page-container {
  max-width: 800px;
  margin: 0 auto;
  padding: 20px;
}

.page-header {
  text-align: center;
  margin-bottom: 20px;
  font-family: sans-serif;
  color: #666;
}

.mushaf-line {
  display: flex;
  justify-content: center;
  direction: rtl;
  font-size: 28px;
  line-height: 2.5;
  margin-bottom: 8px;
  min-height: 48px;
}

.word {
  display: inline-block;
  cursor: pointer;
  padding: 0 2px;
  transition: background-color 0.2s;
}

.word:hover {
  background-color: rgba(0, 123, 255, 0.1);
  border-radius: 4px;
}

.word.loading {
  font-family: 'UthmanicHafs', serif;
  opacity: 0.7;
}

.verse-separator {
  display: inline-block;
  font-family: 'UthmanicHafs', serif;
```

```
      color: #888;
      margin: 0 4px;
    }
  </style>
</head>
<body>
  <div class="page-container">
    <div class="page-header">
      <span id="page-info">Loading...</span>
    </div>
    <div id="mushaf-page"></div>
  </div>

  <script>
    const CDN_BASE = 'https://verses.quran.foundation';
    const API_BASE = 'https://apis.quran.foundation/content/api/v4';
    const loadedFonts = new Set();

    // NOTE: In production, obtain these from your server
    const accessToken = 'YOUR_ACCESS_TOKEN';
    const clientId = 'YOUR_CLIENT_ID';

    /**
     * Load QCF V2 font for a specific Mushaf page
     */
    async function loadPageFont(pageNumber) {
      const fontName = `p${pageNumber}-v2`;
      if (loadedFonts.has(fontName)) return fontName;

      try {
        const fontFace = new FontFace(
          fontName,
          `url('${CDN_BASE}/fonts/quran/hafs/v2/woff2/p${pageNumber}.woff2')`
        );
        fontFace.display = 'block';
        await fontFace.load();
        document.fonts.add(fontFace);
        loadedFonts.add(fontName);
      } catch (error) {
        console.error(`Failed to load font for page ${pageNumber}:`, error);
      }
      return fontName;
    }
```

```
/**
 * Group words by their line number for Mushaf layout
 */
function groupWordsByLine(verses) {
  const lines = new Map();

  // Extract all words from all verses
  verses.forEach(verse => {
    verse.words.forEach(word => {
      const lineKey = `line-${word.line_number}`;

      if (!lines.has(lineKey)) {
        lines.set(lineKey, []);
      }
      lines.get(lineKey).push({
        ...word,
        verseKey: verse.verse_key
      });
    });
  });

  // Convert to sorted array of lines
  return Array.from(lines.entries())
    .sort((a, b) => {
      const lineA = parseInt(a[0].split('-')[1]);
      const lineB = parseInt(b[0].split('-')[1]);
      return lineA - lineB;
    })
    .map(([lineKey, words]) => ({
      lineNumber: parseInt(lineKey.split('-')[1]),
      words
    }));
}

/**
 * Render a single word with QCF font
 */
function renderWord(word, isFontLoaded) {
  const span = document.createElement('span');
  span.className = 'word' + (isFontLoaded ? '' : ' loading');
  span.dataset.page = word.page_number;
  span.dataset.verseKey = word.verseKey;
  span.dataset.position = word.position;
```

```
  // Handle verse end markers with Unicode font
  if (word.char_type_name === 'end') {
    span.classList.add('verse-separator');
    span.textContent = word.text_qpc_hafs;
  } else if (isFontLoaded) {
    // Use QCF glyph - MUST use innerHTML for glyph codes!
    span.style.fontFamily = `p${word.page_number}-v2`;
    span.innerHTML = word.code_v2;
  } else {
    // Fallback to Unicode while font loads
    span.textContent = word.text_qpc_hafs;
  }

  return span;
}

/**
 * Render a Mushaf line (may contain words from multiple verses)
 */
function renderLine(lineData, loadedPages) {
  const lineDiv = document.createElement('div');
  lineDiv.className = 'mushaf-line';
  lineDiv.dataset.lineNumber = lineData.lineNumber;

  lineData.words.forEach(word => {
    const isFontLoaded = loadedPages.has(word.page_number);
    const wordSpan = renderWord(word, isFontLoaded);
    lineDiv.appendChild(wordSpan);
  });

  return lineDiv;
}

/**
 * Fetch and render a Mushaf page
 */
async function renderMushafPage(pageNumber) {
  const container = document.getElementById('mushaf-page');
  const pageInfo = document.getElementById('page-info');

  container.innerHTML = '';
  pageInfo.textContent = `Page ${pageNumber} - Loading...`;

  try {
```

```
// Fetch verses for this page with line_number included
const response = await fetch(
  `${API_BASE}/verses/by_page/${pageNumber}?` +
  `words=true&word_fields=code_v2,text_qpc_hafs,line_number,page_number&mush
  {
    headers: {
      'x-auth-token': accessToken,
      'x-client-id': clientId
    }
  }
);
const data = await response.json();
const verses = data.verses;

if (!verses || verses.length === 0) {
  pageInfo.textContent = `Page ${pageNumber} - No verses found`;
  return;
}

// Get unique page numbers (usually just one, but can span pages)
const pageNumbers = [...new Set(
  verses.flatMap(v => v.words.map(w => w.page_number))
)];

// Group words by line for Mushaf layout
const lines = groupWordsByLine(verses);

// Render with fallback fonts first
const loadedPages = new Set();
lines.forEach(lineData => {
  const lineDiv = renderLine(lineData, loadedPages);
  container.appendChild(lineDiv);
});

pageInfo.textContent = `Page ${pageNumber} - ${verses.length} verses`;

// Load QCF fonts in parallel
await Promise.all(pageNumbers.map(loadPageFont));

// Update words with loaded fonts
pageNumbers.forEach(pn => loadedPages.add(pn));

document.querySelectorAll('.word.loading').forEach(span => {
  const page = parseInt(span.dataset.page);
```

```
          if (loadedPages.has(page) && !span.classList.contains('verse-separator'))
            span.classList.remove('loading');
            span.style.fontFamily = `p${page}-v2`;
            // Re-render with glyph code
            const verseKey = span.dataset.verseKey;
            const position = parseInt(span.dataset.position);

            // Find the original word data
            const verse = verses.find(v => v.verse_key === verseKey);
            const word = verse?.words.find(w => w.position === position);

            if (word && word.code_v2) {
              span.innerHTML = word.code_v2;
            }
          }
        });

      } catch (error) {
        console.error('Failed to render page:', error);
        pageInfo.textContent = `Page ${pageNumber} - Error loading`;
      }
    }

    // Render page 1 (Al-Fatiha starts on page 1)
    renderMushafPage(1);
  </script>
</body>
</html>
```

Key implementation details:

1. **Line Grouping:** Words are grouped by `line_number` to match the physical Mushaf layout
2. **Progressive Loading:** Unicode fallback text is shown immediately, then replaced with QCF glyphs when fonts load
3. **Verse End Markers:** `char_type_name === 'end'` words use Unicode font (renders better)
4. **innerHTML for Glyphs:** QCF `code_v2` values **must** use `innerHTML`, not `textContent`
5. **Cross-Verse Lines:** Each line may contain words from multiple verses—this is handled by grouping

Translation View: Verse-by-Verse Rendering

Translation View displays verses one by one with their translations. No line grouping needed.

Requesting Translations

To include translations in your API response, add the `translations` parameter with comma-separated translation resource IDs:

```
curl -X GET "https://apis.quran.foundation/content/api/v4/verses/by_chapter/1?words=true&word_fields=code_v2,text_qpc_hafs&translations=131,95" \
  -H "x-auth-token: YOUR_ACCESS_TOKEN" \
  -H "x-client-id: YOUR_CLIENT_ID"
```

The response will include a `translations` array for each verse:

```
{
  "verses": [
    {
      "verse_key": "1:1",
      "words": [...],
      "translations": [
        {
          "id": 1338348,
          "resource_id": 95,
          "text": "In the name of Allah, the Merciful, the Compassionate<sup foot_note=\"176997\">1</sup>"
        }
      ]
    }
  ]
}
```

Handling Translation HTML and Footnotes

> ⚠️ **Important:** Translation `text` is returned as **HTML**, not plain text. It may contain footnote markers that need special handling.

Footnote Format:

Footnotes appear as `<sup>` elements with a `foot_note` attribute containing the footnote ID:

```
"text": "Praise<sup foot_note=\"176998\">1</sup> be to Allah, the Lord<sup
foot_note=\"176999\">2</sup> of the entire universe."
```

Rendering Translation Text:

Since the text contains HTML, you must render it using `innerHTML` (or React's `dangerouslySetInnerHTML`):

```
// ✅ Correct - renders HTML including footnote markers
translationDiv.innerHTML = translation.text;

// ❌ Wrong - displays raw HTML tags as text
translationDiv.textContent = translation.text;
```

Fetching Footnote Content:

When a user clicks a footnote, fetch its content using the footnote ID:

```
// Extract footnote ID from the clicked <sup> element
const footnoteId = supElement.getAttribute('foot_note');

// Fetch footnote content
const response = await fetch(
  `${API_BASE}/foot_notes/${footnoteId}`,
  { headers: { 'x-auth-token': accessToken, 'x-client-id': clientId } }
);
const data = await response.json();
// data.footNote.text contains the footnote explanation
```

Complete Footnote Handling Example:

```
function renderTranslation(translation, container) {
  const div = document.createElement('div');
  div.innerHTML = translation.text;

  // Add click handler for footnotes
  div.addEventListener('click', async (event) => {
    const target = event.target;
    if (target.tagName !== 'SUP') return;
```

```
    const footnoteId = target.getAttribute('foot_note');
    if (!footnoteId) return;

    // Fetch and display footnote
    const response = await fetch(`${API_BASE}/foot_notes/${footnoteId}`, {
      headers: { 'x-auth-token': accessToken, 'x-client-id': clientId }
    });
    const data = await response.json();
    showFootnotePopup(data.footNote.text);
  });

  container.appendChild(div);
}
```

Data Flow (Pseudo-code):

```
for each verse in verses:
    render verse.words (Quran text with appropriate font)
    for each translation in verse.translations:
        render translation.text as HTML (not textContent!)
        attach click handlers for footnote <sup> elements
```

Font Files & CDN

CDN Base URL

```
https://verses.quran.foundation/fonts/quran
```

Font File Paths

QCF V1 (604 page files)

```
https://verses.quran.foundation/fonts/quran/hafs/v1/woff2/p{PAGE}.woff2
https://verses.quran.foundation/fonts/quran/hafs/v1/woff/p{PAGE}.woff
https://verses.quran.foundation/fonts/quran/hafs/v1/ttf/p{PAGE}.ttf
```

Replace `{PAGE}` with 1-604.

QCF V2 (604 page files)

```
https://verses.quran.foundation/fonts/quran/hafs/v2/woff2/p{PAGE}.woff2
https://verses.quran.foundation/fonts/quran/hafs/v2/woff/p{PAGE}.woff
https://verses.quran.foundation/fonts/quran/hafs/v2/ttf/p{PAGE}.ttf
```

QCF V4 Tajweed (604 page files)

COLRv1 format (Chrome, Safari, Edge - supports CSS font-palette):

```
https://verses.quran.foundation/fonts/quran/hafs/v4/colrv1/woff2/p{PAGE}.woff2
https://verses.quran.foundation/fonts/quran/hafs/v4/colrv1/woff/p{PAGE}.woff
https://verses.quran.foundation/fonts/quran/hafs/v4/colrv1/ttf/p{PAGE}.ttf
```

OT-SVG format (Firefox dark mode - baked-in colors):

```
https://verses.quran.foundation/fonts/quran/hafs/v4/ot-
svg/light/woff2/p{PAGE}.woff2
https://verses.quran.foundation/fonts/quran/hafs/v4/ot-
svg/dark/woff2/p{PAGE}.woff2
https://verses.quran.foundation/fonts/quran/hafs/v4/ot-
svg/sepia/woff2/p{PAGE}.woff2
```

Unicode Fonts (Single Files)

QPC Hafs / Uthmanic Hafs:

```
https://verses.quran.foundation/fonts/quran/hafs/uthmanic_hafs/UthmanicHafs1Ver18.wo
https://verses.quran.foundation/fonts/quran/hafs/uthmanic_hafs/UthmanicHafs1Ver18.tt
```

IndoPak Nastaleeq:

```
https://verses.quran.foundation/fonts/quran/hafs/nastaleeq/indopak/indopak-
nastaleeq-waqf-lazim-v4.2.1.woff2
https://verses.quran.foundation/fonts/quran/hafs/nastaleeq/indopak/indopak-
nastaleeq-waqf-lazim-v4.2.1.woff
https://verses.quran.foundation/fonts/quran/hafs/nastaleeq/indopak/indopak-
nastaleeq-waqf-lazim-v4.2.1.ttf
```

⚠️ Important: Do Not Store Files Locally

> **We strongly recommend against downloading and storing font files, JSON data, or other assets locally in your application.**
>
> **Why?**
>
> - Font files and data are periodically updated with corrections, improvements, and new features
> - Locally stored files will become outdated without your knowledge
> - You may end up serving incorrect or stale Quranic content to your users
>
> **Best Practice:** Always load fonts and data directly from the CDN at runtime. The CDN is fast, reliable, and ensures your users always receive the latest, most accurate content.

Rendering QCF Fonts (V1, V2, V4)

QCF fonts require special handling because each Mushaf page (1-604) has its own font file.

Step 1: Fetch Verse Data

Request `code_v2` (or `code_v1`), `text_qpc_hafs` (for fallback), and `page_number`:

```
const API_BASE = 'https://apis.quran.foundation/content/api/v4';

async function fetchVerses(chapter, accessToken, clientId) {
  const response = await fetch(
    `${API_BASE}/verses/by_chapter/${chapter}?` +
    `words=true&word_fields=code_v2,text_qpc_hafs&mushaf=1`,
    {
      headers: {
        'x-auth-token': accessToken,
        'x-client-id': clientId
      }
    }
  );
  const data = await response.json();
  return data.verses;
}
```

Step 2: Load Page Fonts Dynamically

```
const loadedFonts = new Set();

async function loadFontForPage(pageNumber, version = 'v2') {
  const fontName = `p${pageNumber}-${version}`;

  if (loadedFonts.has(fontName)) {
    return fontName; // Already loaded
  }

  const fontUrl =
`https://verses.quran.foundation/fonts/quran/hafs/${version}/woff2/p${pageNumber}.wo

  const fontFace = new FontFace(fontName, `url('${fontUrl}')`);
  fontFace.display = 'block';

  await fontFace.load();
  document.fonts.add(fontFace);
  loadedFonts.add(fontName);

  return fontName;
}
```

Step 3: Render Words with Correct Font

> ⚠️ **Critical:** QCF glyph codes contain special Unicode characters that must be rendered using `innerHTML` (or React's `dangerouslySetInnerHTML`), not `textContent`. Using `textContent` will display incorrect characters.

```
async function renderVerse(verse, container) {
  // Get unique page numbers from words
  const pageNumbers = [...new Set(verse.words.map(w => w.page_number))];

  // Load all required fonts
  await Promise.all(pageNumbers.map(page => loadFontForPage(page)));

  // Render each word
  verse.words.forEach(word => {
    const span = document.createElement('span');

    // Handle verse end markers with Unicode font
```

```
    if (word.char_type_name === 'end') {
      span.style.fontFamily = 'UthmanicHafs, serif';
      span.textContent = word.text_qpc_hafs;
    } else {
      span.style.fontFamily = `p${word.page_number}-v2`;
      span.innerHTML = word.code_v2; // MUST use innerHTML for QCF codes!
    }

    container.appendChild(span);
    container.appendChild(document.createTextNode(' ')); // Word separator
```

Step 4: Implement Fallback

Show readable text while fonts load:

```
function renderWordWithFallback(word, fontLoaded) {
  const span = document.createElement('span');

  if (fontLoaded) {
    span.style.fontFamily = `p${word.page_number}-v2`;
    span.innerHTML = word.code_v2;
  } else {
    span.style.fontFamily = 'UthmanicHafs, serif';
    span.textContent = word.text_qpc_hafs; // Fallback text
  }

  return span;
}
```

Complete QCF Implementation

```
<!DOCTYPE html>
<html lang="ar" dir="rtl">
<head>
  <meta charset="UTF-8">
  <title>Quran QCF V2 Rendering</title>
  <style>
    @font-face {
      font-family: 'UthmanicHafs';
      src:
url('https://verses.quran.foundation/fonts/quran/hafs/uthmanic_hafs/UthmanicHafs1Ver
format('woff2');
      font-display: swap;
```

```
    }

    .verse-container {
      font-size: 32px;
      line-height: 2.5;
      direction: rtl;
      text-align: right;
    }

    .word {
      display: inline-block;
    }

    .word.loading {
      font-family: 'UthmanicHafs', serif;
      opacity: 0.7;
    }
  </style>
</head>
<body>
  <div id="quran" class="verse-container"></div>

  <script>
    const loadedFonts = new Set();
    const CDN_BASE = 'https://verses.quran.foundation';

    async function loadPageFont(pageNumber) {
      const fontName = `p${pageNumber}-v2`;
      if (loadedFonts.has(fontName)) return fontName;

      const fontFace = new FontFace(
        fontName,
        `url('${CDN_BASE}/fonts/quran/hafs/v2/woff2/p${pageNumber}.woff2')`
      );
      fontFace.display = 'block';

      try {
        await fontFace.load();
        document.fonts.add(fontFace);
        loadedFonts.add(fontName);
      } catch (error) {
        console.error(`Failed to load font for page ${pageNumber}:`, error);
      }
```

```
  return fontName;
}

// NOTE: In production, obtain these from your server
const API_BASE = 'https://apis.quran.foundation/content/api/v4';
const accessToken = 'YOUR_ACCESS_TOKEN';
const clientId = 'YOUR_CLIENT_ID';

async function fetchAndRenderChapter(chapterNumber) {
  const container = document.getElementById('quran');
  container.innerHTML = 'Loading...';

  // Fetch verses (with authentication)
  const response = await fetch(
    `${API_BASE}/verses/by_chapter/${chapterNumber}?` +
    `words=true&word_fields=code_v2,text_qpc_hafs&mushaf=1`,
    {
      headers: {
        'x-auth-token': accessToken,
        'x-client-id': clientId
      }
    }
  );
  const data = await response.json();

  container.innerHTML = '';

  // Get all unique pages
  const allPageNumbers = new Set();
  data.verses.forEach(verse => {
    verse.words.forEach(word => allPageNumbers.add(word.page_number));
  });

  // Start loading fonts in parallel
  const fontPromises = [...allPageNumbers].map(loadPageFont);

  // Render immediately with fallback
  data.verses.forEach(verse => {
    const verseDiv = document.createElement('div');
    verseDiv.className = 'verse';

    verse.words.forEach(word => {
      const span = document.createElement('span');
      span.className = 'word loading';
```

```
          span.dataset.page = word.page_number;
          span.dataset.codeV2 = word.code_v2;
          span.textContent = word.text_qpc_hafs; // Show fallback first
          verseDiv.appendChild(span);
          verseDiv.appendChild(document.createTextNode(' '));
        });

        container.appendChild(verseDiv);
      });

      // When fonts load, update to QCF rendering
      await Promise.all(fontPromises);

      document.querySelectorAll('.word.loading').forEach(span => {
        span.classList.remove('loading');
        span.style.fontFamily = `p${span.dataset.page}-v2`;
        span.innerHTML = span.dataset.codeV2;
      });
    }

    // Load Surah Al-Fatiha
    fetchAndRenderChapter(1);
  </script>
</body>
</html>
```

Rendering Unicode Fonts

Unicode fonts are simpler - one font file works for all text.

QPC Hafs Implementation

```
<!DOCTYPE html>
<html lang="ar" dir="rtl">
<head>
  <meta charset="UTF-8">
  <title>Quran Unicode Rendering</title>
  <style>
    @font-face {
      font-family: 'UthmanicHafs';
      src:
```

```
url('https://verses.quran.foundation/fonts/quran/hafs/uthmanic_hafs/UthmanicHafs1Ver
format('woff2'),

url('https://verses.quran.foundation/fonts/quran/hafs/uthmanic_hafs/UthmanicHafs1Ver
format('truetype');
      font-display: swap;
    }

    .quran-text {
      font-family: 'UthmanicHafs', 'Traditional Arabic', 'Scheherazade', serif;
      font-size: 28px;
      line-height: 2;
      direction: rtl;
      text-align: right;
    }

    .verse-number {
      font-family: sans-serif;
      font-size: 14px;
      color: #666;
      margin-right: 8px;
    }
  </style>
</head>
<body>
  <div id="quran" class="quran-text"></div>

  <script>
    // NOTE: In production, obtain these from your server
    const API_BASE = 'https://apis.quran.foundation/content/api/v4';
    const accessToken = 'YOUR_ACCESS_TOKEN';
    const clientId = 'YOUR_CLIENT_ID';

    async function fetchAndRender(chapterNumber) {
      const container = document.getElementById('quran');

      const response = await fetch(
        `${API_BASE}/verses/by_chapter/${chapterNumber}?` +
        `words=true&word_fields=text_qpc_hafs`,
        {
          headers: {
            'x-auth-token': accessToken,
            'x-client-id': clientId
          }
```

```
        }
      );
      const data = await response.json();

      data.verses.forEach(verse => {
        const div = document.createElement('div');

        // Combine words into verse text
        const text = verse.words.map(w => w.text_qpc_hafs).join(' ');
        div.textContent = text;

        // Add verse number
        const verseNum = document.createElement('span');
        verseNum.className = 'verse-number';
        verseNum.textContent = `(${verse.verse_key})`;
        div.appendChild(verseNum);

        container.appendChild(div);
      });
    }

    fetchAndRender(1);
  </script>
</body>
</html>
```

IndoPak Nastaleeq Implementation

```
<!DOCTYPE html>
<html lang="ar" dir="rtl">
<head>
  <meta charset="UTF-8">
  <title>Quran IndoPak Rendering</title>
  <style>
    @font-face {
      font-family: 'IndoPak';
      src:
url('https://verses.quran.foundation/fonts/quran/hafs/nastaleeq/indopak/indopak-
nastaleeq-waqf-lazim-v4.2.1.woff2') format('woff2'),

url('https://verses.quran.foundation/fonts/quran/hafs/nastaleeq/indopak/indopak-
nastaleeq-waqf-lazim-v4.2.1.woff') format('woff');
      font-display: swap;
```

```
    }

    .quran-text {
      font-family: 'IndoPak', 'Noto Nastaliq Urdu', serif;
      font-size: 32px;
      line-height: 2.5;
      direction: rtl;
      text-align: right;
    }
  </style>
</head>
<body>
  <div id="quran" class="quran-text"></div>

  <script>
    // NOTE: In production, obtain these from your server
    const API_BASE = 'https://apis.quran.foundation/content/api/v4';
    const accessToken = 'YOUR_ACCESS_TOKEN';
    const clientId = 'YOUR_CLIENT_ID';

    async function fetchAndRender(chapterNumber) {
      const container = document.getElementById('quran');

      // Use mushaf=3 for IndoPak
      const response = await fetch(
        `${API_BASE}/verses/by_chapter/${chapterNumber}?` +
        `words=true&word_fields=text_indopak&mushaf=3`,
        {
          headers: {
            'x-auth-token': accessToken,
            'x-client-id': clientId
          }
        }
      );
      const data = await response.json();

      data.verses.forEach(verse => {
        const div = document.createElement('div');
        const text = verse.words.map(w => w.text_indopak).join(' ');
        div.textContent = text;
        container.appendChild(div);
      });
    }
```

```
    fetchAndRender(1);
  </script>
</body>
</html>
```

Complete Implementation Examples

React Implementation (QCF V2)

```
import React, { useState, useEffect, useCallback } from 'react';

const CDN_BASE = 'https://verses.quran.foundation';
const loadedFonts = new Set();

// Hook to load QCF fonts
function useQcfFontLoader(verses) {
  const [fontsLoaded, setFontsLoaded] = useState(new Set());

  useEffect(() => {
    if (!verses?.length) return;

    const pageNumbers = new Set();
    verses.forEach(verse => {
      verse.words.forEach(word => {
        if (word.page_number) pageNumbers.add(word.page_number);
      });
    });

    const loadFonts = async () => {
      const newlyLoaded = new Set(fontsLoaded);

      await Promise.all([...pageNumbers].map(async (page) => {
        const fontName = `p${page}-v2`;
        if (loadedFonts.has(fontName)) {
          newlyLoaded.add(page);
          return;
        }

        try {
          const fontFace = new FontFace(
            fontName,
            `url('${CDN_BASE}/fonts/quran/hafs/v2/woff2/p${page}.woff2')`
```

```
          );
          fontFace.display = 'block';
          await fontFace.load();
          document.fonts.add(fontFace);
          loadedFonts.add(fontName);
          newlyLoaded.add(page);
        } catch (error) {
          console.error(`Failed to load font for page ${page}:`, error);
        }
      }));

      setFontsLoaded(newlyLoaded);
    };

    loadFonts();
  }, [verses]);

  return fontsLoaded;
}

// Word component
function QuranWord({ word, isFontLoaded }) {
  if (isFontLoaded) {
    return (
      <span
        style={{ fontFamily: `p${word.page_number}-v2` }}
        dangerouslySetInnerHTML={{ __html: word.code_v2 }}
      />
    );
  }

  // Fallback while loading
  return (
    <span style={{ fontFamily: 'UthmanicHafs, serif', opacity: 0.8 }}>
      {word.text_qpc_hafs}
    </span>
  );
}

// Verse component
function Verse({ verse, loadedPages }) {
  return (
    <div style={{
      direction: 'rtl',
```

```
      textAlign: 'right',
      fontSize: '28px',
      lineHeight: 2.5,
      marginBottom: '16px'
    }}>
      {verse.words.map((word, index) => (
        <React.Fragment key={word.id || index}>
          <QuranWord
            word={word}
            isFontLoaded={loadedPages.has(word.page_number)}
          />
          {' '}
        </React.Fragment>
      ))}
      <span style={{ fontSize: '14px', color: '#666' }}>
        ({verse.verse_key})
      </span>
    </div>
  );
}

// API configuration (get tokens from your server in production)
const API_BASE = 'https://apis.quran.foundation/content/api/v4';

// Main component
function QuranChapter({ chapterNumber, accessToken, clientId }) {
  const [verses, setVerses] = useState([]);
  const [loading, setLoading] = useState(true);
  const loadedPages = useQcfFontLoader(verses);

  useEffect(() => {
    async function fetchVerses() {
      setLoading(true);
      try {
        const response = await fetch(
          `${API_BASE}/verses/by_chapter/${chapterNumber}?` +
          `words=true&word_fields=code_v2,text_qpc_hafs&mushaf=1`,
          {
            headers: {
              'x-auth-token': accessToken,
              'x-client-id': clientId
            }
          }
        );
```

```
        const data = await response.json();
        setVerses(data.verses);
      } catch (error) {
        console.error('Failed to fetch verses:', error);
      }
      setLoading(false);
    }

    fetchVerses();
  }, [chapterNumber, accessToken, clientId]);

  if (loading) return <div>Loading...</div>;

  return (
    <div>
      {verses.map(verse => (
        <Verse
          key={verse.id}
          verse={verse}
          loadedPages={loadedPages}
        />
      ))}
    </div>
  );
}

export default QuranChapter;
```

Vue 3 Implementation

```
<template>
  <div class="quran-container">
    <div v-if="loading">Loading...</div>
    <div v-else>
      <div
        v-for="verse in verses"
        :key="verse.id"
        class="verse"
      >
        <span
          v-for="(word, index) in verse.words"
          :key="word.id || index"
          class="word"
```

```
          :style="getWordStyle(word)"
          v-html="getWordText(word)"
        />
        <span class="verse-number">({{ verse.verse_key }})</span>
      </div>
    </div>
  </div>
</template>

<script setup>
import { ref, onMounted, watch } from 'vue';

const props = defineProps({
  chapterNumber: { type: Number, required: true }
});

const CDN_BASE = 'https://verses.quran.foundation';
const loadedFonts = new Set();
const loadedPages = ref(new Set());
const verses = ref([]);
const loading = ref(true);

async function loadPageFont(pageNumber) {
  const fontName = `p${pageNumber}-v2`;
  if (loadedFonts.has(fontName)) return;

  try {
    const fontFace = new FontFace(
      fontName,
      `url('${CDN_BASE}/fonts/quran/hafs/v2/woff2/p${pageNumber}.woff2')`
    );
    fontFace.display = 'block';
    await fontFace.load();
    document.fonts.add(fontFace);
    loadedFonts.add(fontName);
    loadedPages.value = new Set([...loadedPages.value, pageNumber]);
  } catch (error) {
    console.error(`Failed to load font for page ${pageNumber}:`, error);
  }
}

// API configuration (get tokens from your server in production)
const API_BASE = 'https://apis.quran.foundation/content/api/v4';
```

```
async function fetchVerses() {
  loading.value = true;
  try {
    const response = await fetch(
      `${API_BASE}/verses/by_chapter/${props.chapterNumber}?` +
      `words=true&word_fields=code_v2,text_qpc_hafs&mushaf=1`,
      {
        headers: {
          'x-auth-token': props.accessToken,
          'x-client-id': props.clientId
        }
      }
    );
    const data = await response.json();
    verses.value = data.verses;

    // Load fonts for all pages
    const pageNumbers = new Set();
    data.verses.forEach(verse => {
      verse.words.forEach(word => pageNumbers.add(word.page_number));
    });
    await Promise.all([...pageNumbers].map(loadPageFont));
  } catch (error) {
    console.error('Failed to fetch verses:', error);
  }
  loading.value = false;
}

function getWordStyle(word) {
  if (loadedPages.value.has(word.page_number)) {
    return { fontFamily: `p${word.page_number}-v2` };
  }
  return { fontFamily: 'UthmanicHafs, serif', opacity: 0.8 };
}

function getWordText(word) {
  if (loadedPages.value.has(word.page_number)) {
    return word.code_v2;
  }
  return word.text_qpc_hafs;
}

onMounted(fetchVerses);
watch(() => props.chapterNumber, fetchVerses);
```

```
</script>

<style scoped>
@font-face {
  font-family: 'UthmanicHafs';
  src:
url('https://verses.quran.foundation/fonts/quran/hafs/uthmanic_hafs/UthmanicHafs1Ver
format('woff2');
  font-display: swap;
}

.quran-container {
  direction: rtl;
  text-align: right;
}

.verse {
  font-size: 28px;
  line-height: 2.5;
  margin-bottom: 16px;
}

.word {
  display: inline;
}

.verse-number {
  font-family: sans-serif;
  font-size: 14px;
  color: #666;
  margin-right: 8px;
}
</style>
```

Flutter/Dart Implementation

```
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

class QuranVerse {
  final String verseKey;
  final List<QuranWord> words;
```

```
  QuranVerse({required this.verseKey, required this.words});

  factory QuranVerse.fromJson(Map<String, dynamic> json) {
    return QuranVerse(
      verseKey: json['verse_key'],
      words: (json['words'] as List)
          .map((w) => QuranWord.fromJson(w))
          .toList(),
    );
  }
}

class QuranWord {
  final String? textUthmani;
  final String? textQpcHafs;

  QuranWord({this.textUthmani, this.textQpcHafs});

  factory QuranWord.fromJson(Map<String, dynamic> json) {
    return QuranWord(
      textUthmani: json['text_uthmani'],
      textQpcHafs: json['text_qpc_hafs'],
    );
  }
}

class QuranChapterWidget extends StatefulWidget {
  final int chapterNumber;
  final String accessToken;  // Pass from your auth service
  final String clientId;     // Pass from your auth service

  const QuranChapterWidget({
    Key? key,
    required this.chapterNumber,
    required this.accessToken,
    required this.clientId,
  }) : super(key: key);

  @override
  State<QuranChapterWidget> createState() => _QuranChapterWidgetState();
}

class _QuranChapterWidgetState extends State<QuranChapterWidget> {
```

```
static const String apiBase = 'https://apis.quran.foundation/content/api/v4';
List<QuranVerse> verses = [];
bool loading = true;

@override
void initState() {
  super.initState();
  fetchVerses();
}

Future<void> fetchVerses() async {
  final response = await http.get(
    Uri.parse(
      '$apiBase/verses/by_chapter/${widget.chapterNumber}?'
      'words=true&word_fields=text_uthmani,text_qpc_hafs'
    ),
    headers: {
      'x-auth-token': widget.accessToken,
      'x-client-id': widget.clientId,
    },
  );

  if (response.statusCode == 200) {
    final data = json.decode(response.body);
    setState(() {
      verses = (data['verses'] as List)
          .map((v) => QuranVerse.fromJson(v))
          .toList();
      loading = false;
    });
  }
}

@override
Widget build(BuildContext context) {
  if (loading) {
    return const Center(child: CircularProgressIndicator());
  }

  return ListView.builder(
    itemCount: verses.length,
    itemBuilder: (context, index) {
      final verse = verses[index];
      final text = verse.words
```

```
            .map((w) => w.textQpcHafs ?? w.textUthmani ?? '')
            .join(' ');

        return Padding(
          padding: const EdgeInsets.all(16.0),
          child: Directionality(
            textDirection: TextDirection.rtl,
            child: RichText(
              textAlign: TextAlign.right,
              text: TextSpan(
                children: [
                  TextSpan(
                    text: text,
                    style: const TextStyle(
                      fontFamily: 'UthmanicHafs', // Add to pubspec.yaml
                      fontSize: 24,
                      height: 2,
                      color: Colors.black,
                    ),
                  ),
                  TextSpan(
                    text: ' (${verse.verseKey})',
                    style: const TextStyle(
                      fontSize: 12,
                      color: Colors.grey,
                    ),
                  ),
                ],
              ),
            ),
          ),
        );
      },
    );
  }
}
```

Tajweed Color Themes

Tajweed V4 fonts include colored glyphs for Tajweed rules. You need to handle themes.

Browser Support

Browser	Format	Theme Method
Chrome, Edge, Safari	COLRv1	CSS `font-palette`
Firefox	OT-SVG	Separate font files per theme

COLRv1 with CSS Font Palette

```
/* Define palette overrides for each theme */
@font-palette-values --Light {
  font-family: 'p1-v4';
  base-palette: 0;
}

@font-palette-values --Dark {
  font-family: 'p1-v4';
  base-palette: 1;
}

@font-palette-values --Sepia {
  font-family: 'p1-v4';
  base-palette: 2;
}

/* Apply theme */
.quran-text.light {
  font-palette: --Light;
}

.quran-text.dark {
  font-palette: --Dark;
  background: #1a1a1a;
}

.quran-text.sepia {
  font-palette: --Sepia;
  background: #f4ecd8;
}
```

Firefox Dark Mode Handling

```
function getTajweedFontPath(pageNumber, theme) {
  const isFirefox = navigator.userAgent.includes('Firefox');

  if (isFirefox && theme === 'dark') {
    // Use OT-SVG format with baked-in dark colors
    return `https://verses.quran.foundation/fonts/quran/hafs/v4/ot-
svg/dark/woff2/p${pageNumber}.woff2`;
  }

  // Use COLRv1 for other cases
  return
`https://verses.quran.foundation/fonts/quran/hafs/v4/colrv1/woff2/p${pageNumber}.wof
}
```

Complete Tajweed Example

```
<!DOCTYPE html>
<html lang="ar" dir="rtl">
<head>
  <meta charset="UTF-8">
  <title>Quran Tajweed V4</title>
  <style>
    body {
      margin: 0;
      padding: 20px;
      transition: background 0.3s;
    }

    body.dark {
      background: #1a1a1a;
      color: #fff;
    }

    body.sepia {
      background: #f4ecd8;
    }

    .controls {
      margin-bottom: 20px;
    }

    .verse-container {
```

```
      font-size: 32px;
      line-height: 2.5;
      direction: rtl;
      text-align: right;
    }

    /* Font palettes will be generated dynamically */
  </style>
</head>
<body class="light">
  <div class="controls">
    <button onclick="setTheme('light')">Light</button>
    <button onclick="setTheme('dark')">Dark</button>
    <button onclick="setTheme('sepia')">Sepia</button>
  </div>

  <div id="quran" class="verse-container"></div>

  <script>
    const CDN_BASE = 'https://verses.quran.foundation';
    const loadedFonts = new Map(); // page -> fontName
    let currentTheme = 'light';

    function isFirefox() {
      return navigator.userAgent.includes('Firefox');
    }

    function getFontPath(page, theme) {
      if (isFirefox() && theme === 'dark') {
        return `${CDN_BASE}/fonts/quran/hafs/v4/ot-
svg/dark/woff2/p${page}.woff2`;
      }
      return `${CDN_BASE}/fonts/quran/hafs/v4/colrv1/woff2/p${page}.woff2`;
    }

    async function loadTajweedFont(page, theme) {
      const fontName = `p${page}-v4`;
      const cacheKey = `${fontName}-${theme}`;

      if (loadedFonts.has(cacheKey)) return fontName;

      const fontFace = new FontFace(fontName, `url('${getFontPath(page,
theme)}')`);
      fontFace.display = 'block';
```

```
      await fontFace.load();
      document.fonts.add(fontFace);
      loadedFonts.set(cacheKey, fontName);

      // Add font palette CSS
      addFontPalette(fontName);

      return fontName;
    }

    function addFontPalette(fontFamily) {
      const style = document.createElement('style');
      style.textContent = `
        @font-palette-values --Light-${fontFamily} {
          font-family: '${fontFamily}';
          base-palette: 0;
        }
        @font-palette-values --Dark-${fontFamily} {
          font-family: '${fontFamily}';
          base-palette: 1;
        }
        @font-palette-values --Sepia-${fontFamily} {
          font-family: '${fontFamily}';
          base-palette: 2;
        }
      `;
      document.head.appendChild(style);
    }

    function setTheme(theme) {
      currentTheme = theme;
      document.body.className = theme;

      // Update font palettes on existing words
      document.querySelectorAll('.word').forEach(el => {
        const fontFamily = el.dataset.fontFamily;
        if (fontFamily) {
          el.style.fontPalette = `--${theme.charAt(0).toUpperCase() +
theme.slice(1)}-${fontFamily}`;
        }
      });

      // For Firefox dark mode, reload fonts
```

```
  if (isFirefox() && theme === 'dark') {
    reloadFontsForTheme(theme);
  }
}

async function reloadFontsForTheme(theme) {
  const words = document.querySelectorAll('.word');
  const pages = new Set();
  words.forEach(w => pages.add(parseInt(w.dataset.page)));

  await Promise.all([...pages].map(p => loadTajweedFont(p, theme)));
}

// NOTE: In production, obtain these from your server
const API_BASE = 'https://apis.quran.foundation/content/api/v4';
const accessToken = 'YOUR_ACCESS_TOKEN';
const clientId = 'YOUR_CLIENT_ID';

async function fetchAndRender(chapter) {
  const container = document.getElementById('quran');
  container.innerHTML = 'Loading...';

  const response = await fetch(
    `${API_BASE}/verses/by_chapter/${chapter}?` +
    `words=true&word_fields=code_v2,text_qpc_hafs&mushaf=19`,
    {
      headers: {
        'x-auth-token': accessToken,
        'x-client-id': clientId
      }
    }
  );
  const data = await response.json();

  const pages = new Set();
  data.verses.forEach(v => v.words.forEach(w => pages.add(w.page_number)));

  await Promise.all([...pages].map(p => loadTajweedFont(p, currentTheme)));

  container.innerHTML = '';

  data.verses.forEach(verse => {
    const div = document.createElement('div');
```

```
        verse.words.forEach(word => {
          const span = document.createElement('span');
          span.className = 'word';
          span.dataset.page = word.page_number;
          span.dataset.fontFamily = `p${word.page_number}-v4`;
          span.style.fontFamily = `p${word.page_number}-v4`;
          span.style.fontPalette = `--${currentTheme.charAt(0).toUpperCase() +
currentTheme.slice(1)}-p${word.page_number}-v4`;
          span.innerHTML = word.code_v2;
          div.appendChild(span);
          div.appendChild(document.createTextNode(' '));
        });

        container.appendChild(div);
      });
    }

    fetchAndRender(1);
  </script>
</body>
</html>
```

Font Scaling

When implementing font size controls in your application, consider these guidelines based on how Quran.com handles font scaling.

Recommended Scale System

Use a 10-level scale for Quran text size:

Level	Description	Suggested Use Case
1-3	Small sizes	Mobile-optimized, compact reading
3	**Default**	Balanced default for most users
4-5	Medium sizes	Comfortable extended reading

Level	Description	Suggested Use Case
6-10	Large sizes	Accessibility, presentations

Responsive Font Sizing

Use viewport-relative units for responsive scaling:

```
/* Mobile: use viewport width */
.quran-text-scale-3 {
  font-size: 5.3vw;
}

/* Tablet/Desktop: use viewport height */
@media (min-width: 768px) {
  .quran-text-scale-3 {
    font-size: 3.2vh;
  }
}
```

Font-Specific Scale Values

Different fonts require different scale values for consistent visual appearance:

QCF V2 (`code_v2`) recommended sizes:

Scale	Mobile	Tablet/Desktop
1	4vw	2.9vh
3	5.3vw	3.2vh
5	10vw	3.7vh
10	15vw	11vh

QPC Hafs (Unicode) recommended sizes:

Scale	Mobile	Tablet/Desktop
1	4vw	3.2vh
3	5vw	4vh
5	11vw	4.4vh
10	16vw	10.27vh

Layout Behavior Changes

When implementing a Reading View that aims to replicate physical Mushaf pages, consider these layout behaviors:

Maintaining Mushaf Page Boundaries

At smaller font scales (1-3), you can maintain **physical Mushaf page boundaries**:

- Each line matches the same boundaries as the physical Mushaf
- Page breaks align with printed Mushaf pages
- Words don't overflow their designated line positions

This creates an interleaved reading experience that closely mirrors reading from a physical Quran.

Big Text Layout Mode

At larger font scales (4+), it becomes impossible to maintain exact print fidelity. Consider switching to a relaxed layout:

Aspect	Normal Layout (Scale 1-3)	Big Text Layout (Scale 4+)
Line boundaries	Strictly maintained	Relaxed for readability
Page fidelity	Matches physical Mushaf	May differ from print
Word wrapping	Preserves Mushaf line breaks	Allows natural wrapping

Aspect	Normal Layout (Scale 1-3)	Big Text Layout (Scale 4+)
Primary goal	Authenticity	Accessibility

Implementation tip: Track when users select large font sizes and adjust your layout constraints accordingly:

```
const isBigTextLayout = fontScale > 3;

if (isBigTextLayout) {
  // Relax line width constraints
  // Allow natural word wrapping
  // Prioritize readability over print fidelity
}
```

Best Practices

1. Always Use Fallback Fonts

```
.quran-text {
  font-family: 'UthmanicHafs', 'Traditional Arabic', 'Scheherazade New', serif;
}
```

2. Preload Critical Fonts

```
<link rel="preload"
href="https://verses.quran.foundation/fonts/quran/hafs/uthmanic_hafs/UthmanicHafs1Ve
as="font" type="font/woff2" crossorigin>
```

3. Use font-display: swap

```
@font-face {
  font-family: 'UthmanicHafs';
  src: url('...') format('woff2');
```

```
  font-display: swap; /* Show fallback immediately, swap when loaded */
}
```

4. Load Fonts On-Demand

Don't load all 604 QCF fonts upfront. Load only fonts for visible pages:

```
// Good: Load only needed pages
const visiblePages = getVisiblePageNumbers();
await Promise.all(visiblePages.map(loadPageFont));

// Bad: Load all fonts
for (let i = 1; i <= 604; i++) {
  await loadPageFont(i);
}
```

5. Cache Loaded Fonts

```
const loadedFonts = new Set();

async function loadFontForPage(page) {
  const fontName = `p${page}-v2`;
  if (loadedFonts.has(fontName)) return; // Already loaded

  // Load font...
  loadedFonts.add(fontName);
}
```

6. Handle RTL Properly

```
<html lang="ar" dir="rtl">
```

```
.quran-text {
  direction: rtl;
  text-align: right;
  unicode-bidi: bidi-override;
}
```

7. Request Only Needed Fields

```
// Good: Request only what you need
const response = await fetch(url + '&word_fields=code_v2');

// Bad: Request everything
const response = await fetch(url +
'&word_fields=code_v1,code_v2,text_uthmani,text_indopak,...');
```

Troubleshooting

Fonts Not Loading

Symptom: Text appears in system Arabic font, not Quran font.

Solutions:

1. Check browser console for CORS errors
2. Verify font URL is correct
3. Check if FontFace API is supported
4. Ensure font file exists at CDN path

```
// Debug font loading
const fontFace = new FontFace('test', `url('${fontUrl}')`);
fontFace.load()
  .then(() => console.log('Font loaded successfully'))
  .catch(err => console.error('Font load failed:', err));
```

QCF Glyphs Show as Squares

Symptom: Text shows placeholder squares instead of Arabic.

Solutions:

1. Ensure correct font file is loaded for the page number
2. Verify `code_v2` field is being used (not `text_qpc_hafs`)
3. Check `font-family` matches the loaded font name

Wrong Mushaf Layout

Symptom: Verse breaks or word order seems wrong.

Solutions:

1. Verify `mushaf` parameter matches font type
2. For IndoPak, check 15 vs 16 lines setting

Firefox V1 Mushaf Spacing Issues

Symptom: Words in QCF V1 Mushaf appear too close together or overlap in Firefox.

Cause: Firefox has a `word-spacing` bug with QCF V1 fonts at smaller font scales.

Solution: Add extra spacing when using V1 Mushaf with font scale less than 6:

```
/* Firefox V1 spacing fix */
@-moz-document url-prefix() {
  .qcf-v1-word {
    /* Add extra space after each word at smaller scales */
    margin-inline-end: 0.1em; /* Adjust as needed */
  }
}

/* Or use word-spacing with a fallback */
.qcf-v1-container {
  word-spacing: 0.05em; /* Helps Firefox spacing */
}
```

```
// Detection approach
const isFirefox = navigator.userAgent.includes('Firefox');
const isV1Mushaf = mushafId === 2; // QCFV1
const needsSpacingFix = isFirefox && isV1Mushaf && fontScale < 6;

if (needsSpacingFix) {
  // Add extra spacing between words
  wordElement.style.marginInlineEnd = '0.1em';
}
```

Tajweed Colors Not Showing

Symptom: Tajweed text is black instead of colored.

Solutions:

1. For non-Firefox: Check `font-palette` CSS is applied
2. For Firefox dark mode: Use OT-SVG font files
3. Verify V4 font files are loaded (not V2)